

CODE SPORT



In this month's column, we take a quick look at how to do effective code reviews.

For the past couple of months, we have been discussing interview questions. Now, we take a break and focus on how to do effective code reviews.

As you would know, being a software developer involves spending considerable time reviewing the code written by your peers. While this is an important part of a software developer's job, it's not something that every developer is fond of doing. After all, reading other people's code is not easy. Consider that you are asked to review a defect fix that one of your colleagues has submitted for review. Now you first need to understand the original software defect which was reported by the customer, figure out how the fix addresses that issue, then mentally evaluate whether this is the best way of addressing that defect and if so, look through the code to see whether the implementation is correct and does not introduce any further defects.

Doing reviews takes as much time (if not more) as writing code does. However, sadly, this aspect is not recognised in many software development organisations. Typically, your manager would expect you to complete reviewing 2000-5000 lines of code in a day, as a routine effort, whereas no one would demand that you write production quality bullet-proof code of 2000-5000 lines in a single day. Being a code reviewer makes you responsible for making sure that no defects escape your review. Hence, in this column, we will look at some of the effective ways of reviewing code.

Let's assume that you have been asked to review a major feature of 1000 lines and you are midway through the code review. At which point you come across the following code snippet:

```
int foo(int id...)
{
    //the code accepts a id which is 23 bits, hence its max
    value is 2^23-1
    Int limit = 2^23 -1;
    assert (id < limit);
```

```
.....
//do something else and compute result based on id
}
```

Now assuming that the above code snippet was part of the code you are reviewing, would you find anything wrong with it? Well, if you look closely and carefully at the statement which computes the limit, you'll definitely find something amiss. What is the value computed by this statement? In C language, operator '^' does not represent the exponent operator. It actually represents the XOR operator. So while the developer's original intent was to compute the value of 2 raised to the power of 23, the actual code computes the value as 2 XOR 23, and hence the limit gets set to 21 and not to 8388607, which was the actual intended value.

Whenever this code is presented to developers, I find that most of them do not find anything wrong with the code. It becomes all the more difficult to spot an error when it is buried among the thousands of lines of code you are reviewing. The reason is that we read the comment above the wrong code and validate the comment. The comment is in English and not in 'C' programming language. Hence, we interpret the comment as is, and validate the statement the developer was making. The intent is correct: that he wants to check whether the value is less than (2^{23}). Now when he actually implements this intent into a 'C' programming language statement, the coding error creeps in. Since our brain has already validated the previous statement and found it to be correct, most of us tend to skip over or quickly glance over the next statement. Hence we end up overlooking the coding mistake. When the same code snippet is presented without the comment, I have found reviewers questioning the author of the code as to how the limit value is assumed, and whether it is meant to be 21. Without the comment above it, most reviewers definitely pay more attention to that line and hence verify its intent with the author of the code.